Astrofísica Moderna

Prof. Germán Chaparro

Proyecto 1

Sintetización Monte Carlo de la silueta de un agujero negro con Ray Tracing

---

Nombre: Juan Felipe Garcés Carvajal
Cédula: 1011511457

In [ ]:
```
#@title Importación de Librerías
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from scipy.integrate import odeint
```

Parte 1: Inicialización de los Rayos

In [ ]:
```
N = 500 # número de rayos
R = 70  # radio del círculo
```

In [ ]:
```
np.random.seed(42) # esto permite que las posiciones generadas siempre sean las mis
```

In [ ]:
```
u = np.random.uniform(0, 1, N)
theta_rand = np.random.uniform(0, 2*np.pi, N)
r0 = R * np.sqrt(u)
```

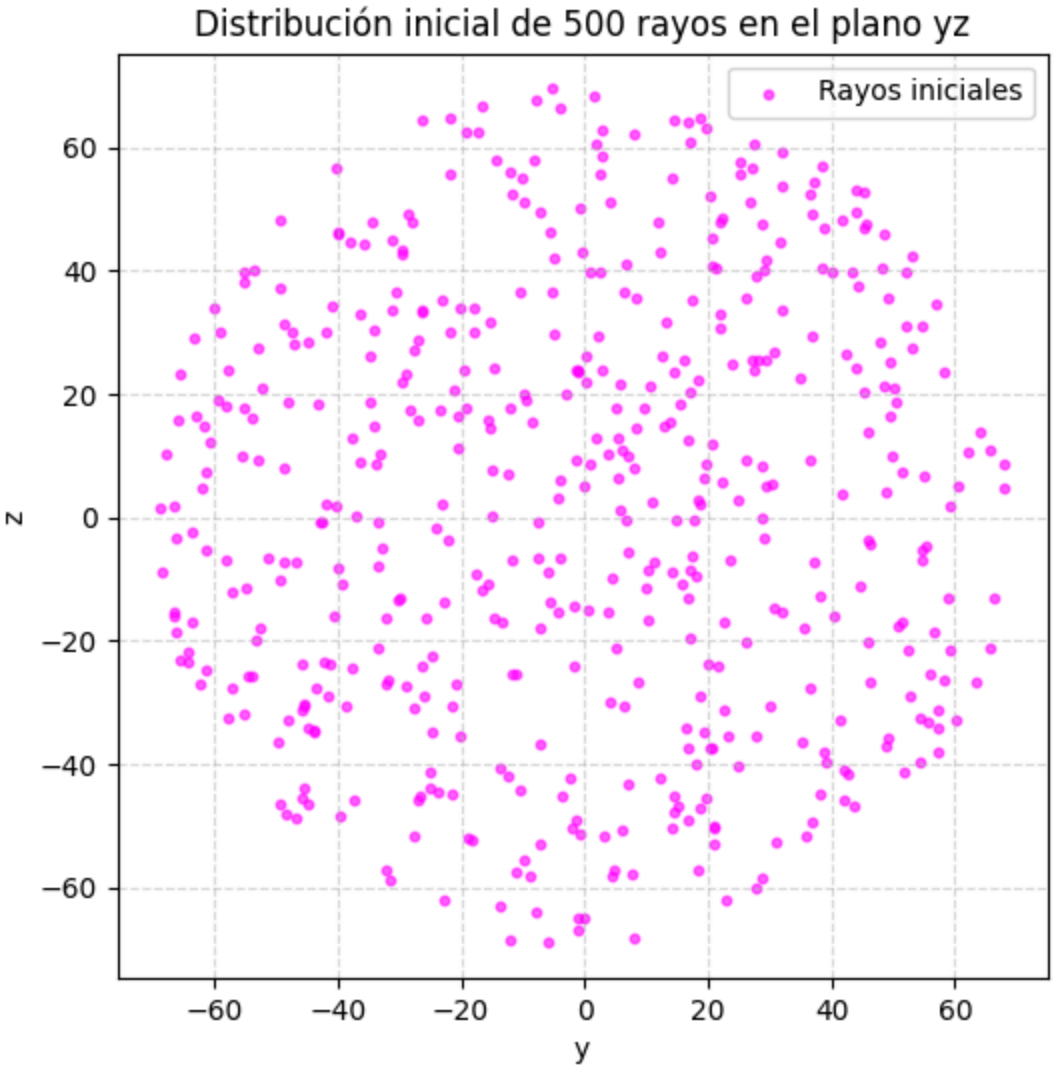
Se utiliza `u` (uniforme en [0,1]) para aplicar el método de la transformada inversa. Dado que el área de un círculo crece con r^2, generar el radio uniformemente concentraría los puntos en el centro. Usando $(r = R\sqrt{u})$, garantizamos que la densidad de puntos sea constante en toda el área del círculo, logrando una distribución espacial uniforme.

In [ ]:
```
y0_i = r0 * np.cos(theta_rand)
z0_i = r0 * np.sin(theta_rand)
```

In [ ]:
```
# debemos conservar theta_0 para deshacer la rotación más tarde
theta0 = np.arctan2(z0_i, y0_i)
```

Visualización

In [ ]:
```
plt.style.use('default') # tuve que añadir esto porque las últimas graficas son neg
plt.figure(figsize=(6, 6))
plt.scatter(y0_i, z0_i, s=10, alpha=0.6, color='magenta', label='Rayos iniciales')
plt.title(f'Distribución inicial de {N} rayos en el plano yz')
plt.xlabel('y')
plt.ylabel('z')
plt.axis('equal') # para que el círculo no se vea como una elipse
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```



Parte 2: Integración de los Rayos

Las ecuaciones de Eikonal se tomaron del documento del proyecto.

In [ ]:
```
A = 10.0 # para (x^2 + y^2)^1/2 < A

def refindex(x, y):
  eps = 1e-6 # corrección para evitar singularidades numéricas
  rp0 = np.sqrt(x**2 + y**2)
  n = 1 / (1 - A / (rp0 + eps))
  fac = np.abs((1 - 9*(A / rp0)**2 / 8))
  nx = -fac * n**2 * A * x / (rp0 + eps)**3
  ny = -fac * n**2 * A * y / (rp0 + eps)**3
  return [n, nx, ny]

def flow_deriv(x_y_z, tspan):
  x, y, z, w = x_y_z
  [n, nx, ny] = refindex(x, y)
  yp = np.zeros(shape=(4,))
  yp[0] = z / n
  yp[1] = w / n
  yp[2] = nx
  yp[3] = ny
  return yp
```

In [ ]:
```
delta_t = 1
tspan = np.arange(0,400,delta_t)

xstart = -100.0 # posición x inicial para todos los rayos
xmax_limit = 300.0 # límite para detener rayos que escapan
ymax_limit = 300.0
```

In [ ]:
```
# guardamos las trayectorias y el estado final
trayectorias = []
estados_finales = []
```

In [ ]:
```
for i in range(len(r0)):
  y0 = [xstart, r0[i], 1.0, 0.0] # estado inicial x0 = -100 y y0 = r0

  sol = odeint(flow_deriv, y0, tspan) # integración

  x_traj = sol[:, 0]
  y_traj = sol[:, 1]
  r_traj = np.sqrt(x_traj**2 + y_traj**2) # división de elementos de la solución

  indices_bh = np.where(r_traj < A)[0] # aplicamos la condición r < A

  if len(indices_bh) > 0:
    idx_corte = indices_bh[0]
    trayectoria_truncada = sol[:idx_corte+1]
    estado = 'Agujero Negro'

  else:

    indices_escape = np.where(
            (np.abs(x_traj) > xmax_limit) | #  x > xmax o (| significa "o") y > yma
            (np.abs(y_traj) > ymax_limit)
        )[0]
```

```
    if len(indices_escape) > 0:

      idx_corte = indices_escape[0]
      trayectoria_truncada = sol[:idx_corte+1] # si la trajectoria escapó, se corta
      estado = 'Escapó'

    else:
      trayectoria_truncada = sol
      estado = 'Atrapado/Detenido' # ni escapa ni se cae al agujero
  trayectorias.append(trayectoria_truncada)
  estados_finales.append(estado)
```

```
/tmp/ipykernel_7300/4157571669.py:4: ODEintWarning: Excess work done on this call (p
erhaps wrong Dfun type). Run with full_output = 1 to get quantitative information.
  sol = odeint(flow_deriv, y0, tspan) # integración
/tmp/ipykernel_7300/4157571669.py:8: RuntimeWarning: overflow encountered in square
  r_traj = np.sqrt(x_traj**2 + y_traj**2) # división de elementos de la solución
```

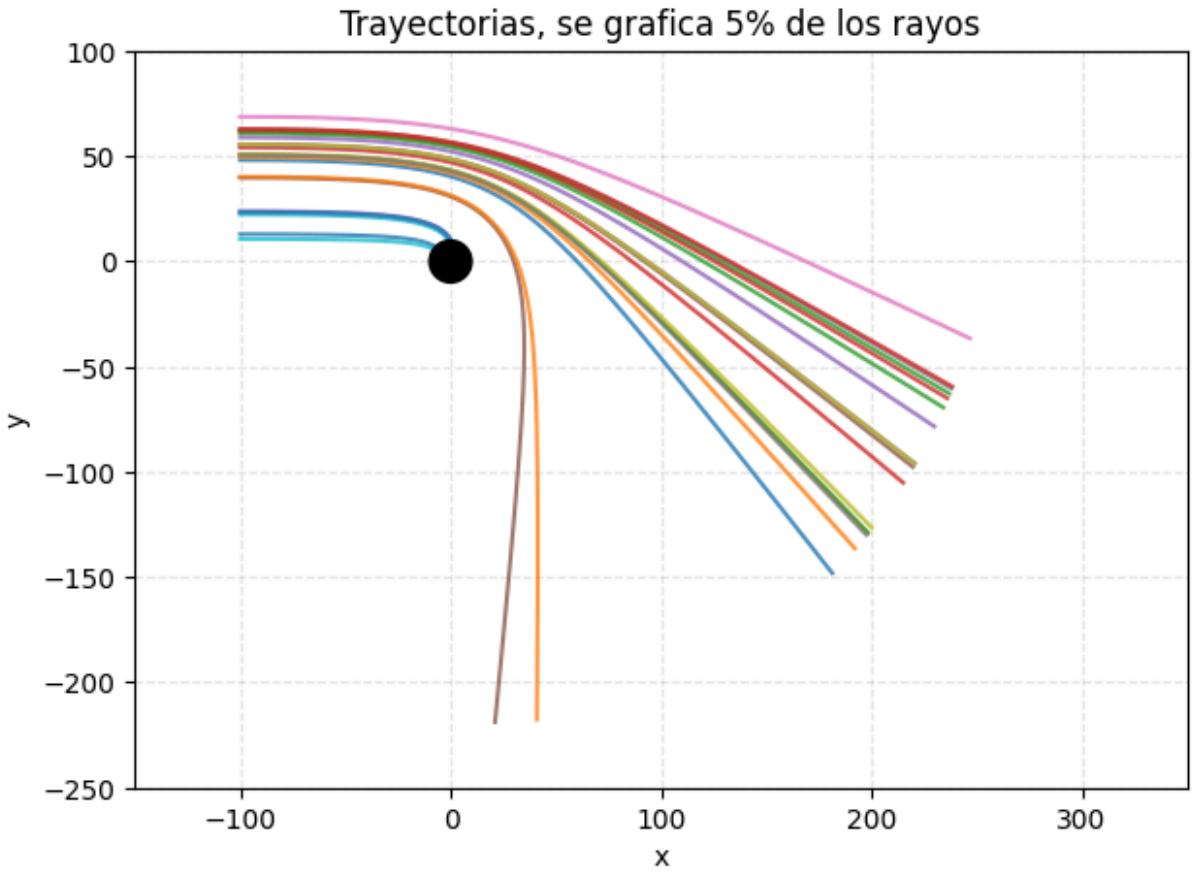
In [ ]:
```
plt.figure(figsize=(7, 6))

# graficamos 5% de los rayos para no saturar la imagen
porcentaje = 0.05
n_graficar = max(1, int(len(trayectorias) * porcentaje))
indices_graficar = np.random.choice(len(trayectorias), n_graficar, replace=False)

for idx in indices_graficar:
    traj = trayectorias[idx]
    plt.plot(traj[:, 0], traj[:, 1], linewidth=1.5, alpha=0.8)

circulo_bh = plt.Circle((0, 0), A, color='black', zorder=10) # agujero negro (círcu
plt.gca().add_patch(circulo_bh)

plt.title(f'Trayectorias, se grafica {porcentaje*100:.0f}% de los rayos')
plt.xlabel('x')
plt.ylabel('y')
plt.xlim(-150, 350)
plt.ylim(-250, 100)
plt.grid(True, linestyle='--', alpha=0.3)
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```



Parte 3: Construcción del Disco

In [ ]:
```
from scipy.spatial.transform import Rotation as R # módulo de rotación sugerido
```

In [ ]:
```
Nd = 5000
Rin = 15.0 # radio interno
Rout = 50.0 # radio externo
```

In [ ]:
```
np.random.seed(123)
```

In [ ]:
```
u_disk = np.random.uniform(0, 1, Nd) # de la misma forma que al principio, la distr
r_disk = np.sqrt(Rin**2 + u_disk * (Rout**2 - Rin**2))
theta_disk = np.random.uniform(0, 2*np.pi, Nd)

x_disk = r_disk * np.cos(theta_disk) # coordenadas cartesianas de los puntos
y_disk = r_disk * np.sin(theta_disk)
z_disk = np.zeros(Nd)
```

In [ ]:
```
dave = np.sqrt(np.pi * Rout**2 / Nd)
print(f"Espaciamiento promedio entre puntos del disco (dave): {dave:.4f}")
```

```
Espaciamiento promedio entre puntos del disco (dave): 1.2533
```

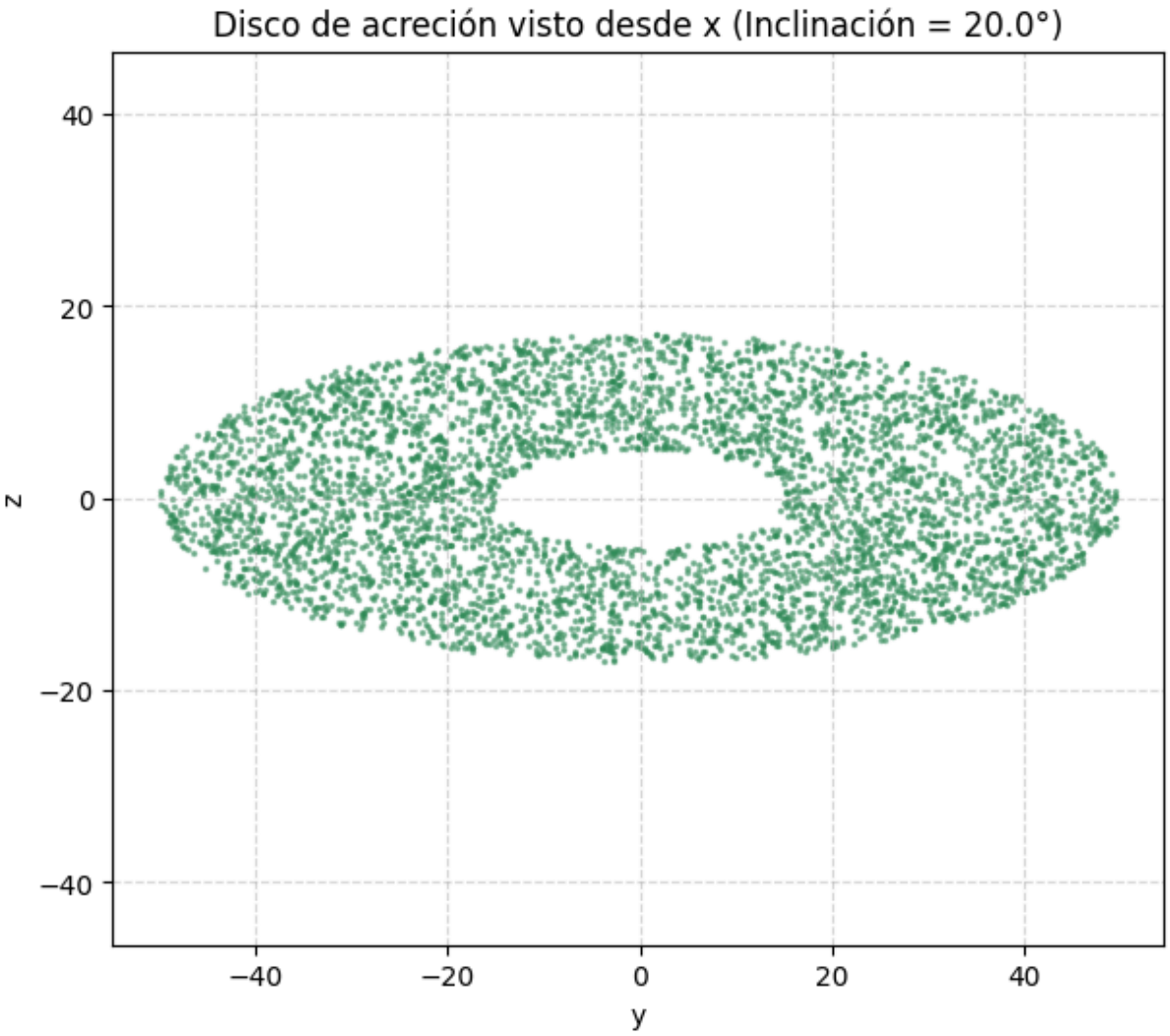
In [ ]:
```
i_deg = 20.0
i_rad = np.radians(i_deg)
```

In [ ]:
```
rot = R.from_euler('y', i_rad) # matriz de rotación
```

In [ ]:
```
# ROTACIÖN
puntos_disco = np.column_stack((x_disk, y_disk, z_disk))
puntos_disco_inclinados = rot.apply(puntos_disco)
```

In [ ]:
```
x_disk_inc = puntos_disco_inclinados[:, 0]
y_disk_inc = puntos_disco_inclinados[:, 1]
z_disk_inc = puntos_disco_inclinados[:, 2] # nuevas coordenadas
```

In [ ]:
```
plt.figure(figsize=(7, 6))
plt.scatter(y_disk_inc, z_disk_inc, s=2, alpha=0.5, color='seagreen')
plt.title(f'Disco de acreción visto desde x (Inclinación = {i_deg}°)')
plt.xlabel('y')
plt.ylabel('z')
plt.axis('equal')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



Parte 4: Intersecciones entre los Rayos y el Disco

In [ ]:
```
def Rx(theta): # matriz de rotación
    c = np.cos(theta)
    s = np.sin(theta)
    return np.array([
        [1, 0, 0],
        [0, c, -s],
        [0, s, c]
    ])
```

In [ ]:
```
N_rayos = len(trayectorias)
resultados = []  # guardaremos (y0, z0, radio_interseccion)
```

In [ ]:
```
for i in range(N_rayos):
  traj_2d = trayectorias[i]  # trayectoria 2D del rayo

  x_traj = traj_2d[:, 0]
  y_traj = traj_2d[:, 1]
  z_traj = np.zeros_like(x_traj)  # z=0 (plano xy)

  puntos_rayo_2d = np.column_stack((x_traj, y_traj, z_traj)) # todos los rayos en u

  # matrices de rotación con theta0 específico (el de la parte 1)
  theta_rayo = theta0[i]
  rot_matrix = Rx(theta_rayo)

  # rotación de los rayos
  puntos_rayo_3d = puntos_rayo_2d @ rot_matrix.T

  distancias_minimas = np.full(len(puntos_rayo_3d), np.inf)  # arreglo con len(punt

  for j, punto_rayo in enumerate(puntos_rayo_3d):
    diff = puntos_disco_inclinados - punto_rayo
    distancias = np.sqrt(np.sum(diff**2, axis=1))

    distancias_minimas[j] = np.min(distancias) # distancia mínima para este punto d

  # primer punto del rayo que esté lo suficientemente cerca del disco
  interseccion_idx = np.where(distancias_minimas < dave)[0] # se usa el espaciamien

  if len(interseccion_idx) > 0:
    idx_int = interseccion_idx[0]
    punto_int = puntos_rayo_3d[idx_int]

    r_int = np.sqrt(punto_int[0]**2 + punto_int[1]**2 + punto_int[2]**2) # distanci

        # Guardar el resultado
    resultados.append((y0_i[i], z0_i[i], r_int))
  else:
```

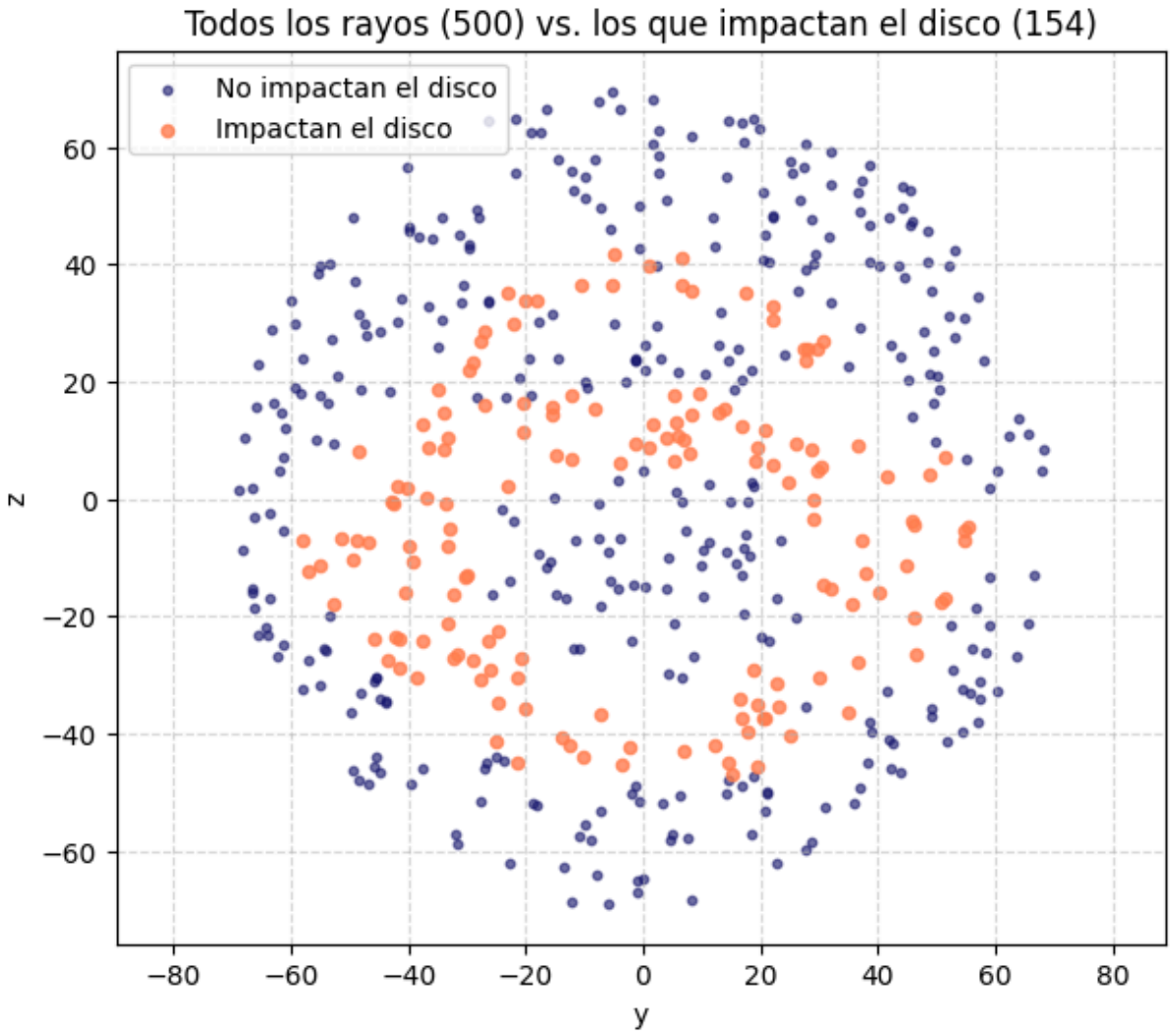
```
        # No hubo intersección
    resultados.append((y0_i[i], z0_i[i], -999))
```

In [ ]:
```
resultados = np.array(resultados)
y0_plot = resultados[:, 0]
z0_plot = resultados[:, 1]
r_int_plot = resultados[:, 2]
```

In [ ]:
```
mask_impacto = r_int_plot > 0
mask_no_impacto = ~mask_impacto

plt.figure(figsize=(7, 6))
plt.scatter(y0_plot[mask_no_impacto], z0_plot[mask_no_impacto],
            s=10, color='midnightblue', alpha=0.6, label='No impactan el disco')
plt.scatter(y0_plot[mask_impacto], z0_plot[mask_impacto],
            s=20, color='coral', alpha=0.8, label='Impactan el disco')

plt.title(f'Todos los rayos ({N_rayos}) vs. los que impactan el disco ({np.sum(mask
plt.xlabel('y')
plt.ylabel('z')
plt.axis('equal')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend()
plt.show()
```



Parte 5: Síntedsis de la Imagen

Hay que repetir lo anterior con N > 10000 rayos (si se puede en mi portatil). Para ello se creará una función que haga todo (puntos 1 al 3) solo con N y la inclinación como argumentos

In [ ]:
```
from scipy.spatial import cKDTree
import time
```

cKDTree sirve para calcular más facilmente las distancias entre los puntos del disco y los rayos, reduciendo el tiempo de computación.

In [ ]:
```
def simular_agujero_negro(N_rayos, inclinacion_grados):
  print(f"Simulación: N={N_rayos}, Inclinación={inclinacion_grados}°")
  start_total = time.time()

  np.random.seed(42)

  R_pantalla = 70.0
```

```
u = np.random.uniform(0, 1, N_rayos)
theta_rand = np.random.uniform(0, 2*np.pi, N_rayos)
r0 = R_pantalla * np.sqrt(u)
y0_inicial = r0 * np.cos(theta_rand)
z0_inicial = r0 * np.sin(theta_rand)
theta0 = np.arctan2(z0_inicial, y0_inicial)
theta0 = np.where(theta0 < 0, theta0 + 2*np.pi, theta0)

tspan = np.arange(0, 401, 2.0)
xstart = -100.0
xmax_limit = 300.0
ymax_limit = 300.0

trayectorias = []

start_int = time.time()
for i in range(N_rayos):
  estado_inicial = [xstart, r0[i], 1.0, 0.0]
  sol = odeint(flow_deriv, estado_inicial, tspan)

  x_traj = sol[:, 0]
  y_traj = sol[:, 1]
  r_traj = np.sqrt(x_traj**2 + y_traj**2)

  indices_bh = np.where(r_traj < A)[0]

  if len(indices_bh) > 0:
    idx_corte = indices_bh[0]
    trayectorias.append(sol[:idx_corte+1])
  else:
    indices_escape = np.where((np.abs(x_traj) > xmax_limit) | (np.abs(y_traj) > y
    if len(indices_escape) > 0:
      idx_corte = indices_escape[0]
      trayectorias.append(sol[:idx_corte+1])
    else:
      trayectorias.append(sol)
print(f"Integración completada en {time.time() - start_int:.2f} s")

Nd = 5000
Rin = 15.0
Rout = 50.0

u_disk = np.random.uniform(0, 1, Nd)
r_disk = np.sqrt(Rin**2 + u_disk * (Rout**2 - Rin**2))
theta_disk = np.random.uniform(0, 2*np.pi, Nd)

x_disk = r_disk * np.cos(theta_disk)
y_disk = r_disk * np.sin(theta_disk)
z_disk = np.zeros(Nd)

dave = np.sqrt(np.pi * Rout**2 / Nd)

i_rad = np.radians(inclinacion_grados)
rot = R.from_euler('y', i_rad)
puntos_disco = np.column_stack((x_disk, y_disk, z_disk))
puntos_disco_inclinados = rot.apply(puntos_disco)
```

```
tree = cKDTree(puntos_disco_inclinados)

resultados = np.zeros((N_rayos, 3))

start_intersect = time.time()
for i in range(N_rayos):
  traj_2d = trayectorias[i]
  x_traj = traj_2d[:, 0]
  y_traj = traj_2d[:, 1]
  z_traj = np.zeros_like(x_traj)

  puntos_rayo_2d = np.column_stack((x_traj, y_traj, z_traj))

  theta_rayo = theta0[i]
  rot_matrix = Rx(theta_rayo)
  puntos_rayo_3d = puntos_rayo_2d @ rot_matrix.T

  distancias, indices = tree.query(puntos_rayo_3d, k=1)

  interseccion_idx = np.where(distancias < dave)[0]

  if len(interseccion_idx) > 0:
    idx_int = interseccion_idx[0]
    punto_int = puntos_rayo_3d[idx_int]
    r_int = np.sqrt(punto_int[0]**2 + punto_int[1]**2 + punto_int[2]**2)
    resultados[i] = (y0_inicial[i], z0_inicial[i], r_int)
  else:
    resultados[i] = (y0_inicial[i], z0_inicial[i], -999)

print(f"Intersecciones completadas en {time.time() - start_intersect:.2f} s")

y0_plot = resultados[:, 0]
z0_plot = resultados[:, 1]
r_int_plot = resultados[:, 2]
mask_impacto = r_int_plot > 0
mask_no_impacto = ~mask_impacto

plt.style.use('dark_background')  # cambia el estilo global a fondo negro. solo m
                                  # ademas, esto (posiblemente)

plt.figure(figsize=(6, 6))

scatter = plt.scatter(y0_plot[mask_impacto], z0_plot[mask_impacto], c=r_int_plot[

plt.colorbar(scatter, label='Distancia radial al agujero negro (r)')
plt.title(f'Silueta del Agujero Negro (N={N_rayos}, Inclinación={inclinacion_grad
plt.xlabel('y', color='white')
plt.ylabel('z', color='white')
plt.axis('equal')
plt.grid(False)

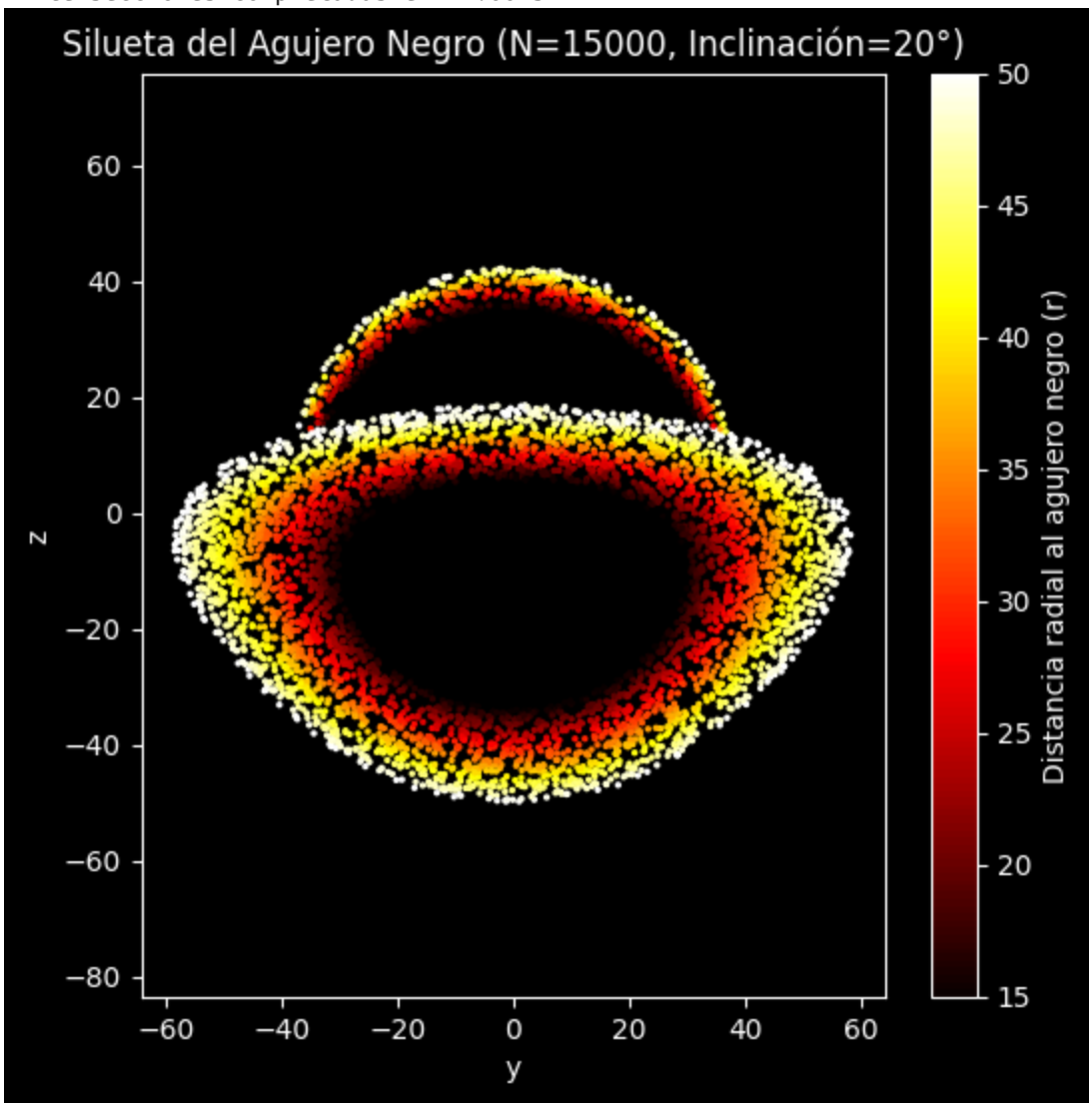
ax = plt.gca()
ax.set_facecolor('black') # los ejes deben ser negros
ax.tick_params(colors='white')
```

```
  plt.show()
  plt.style.use('default') # restaura el estilo de gráficas para que las anteriores
  print(f"Simulación total completada en {time.time() - start_total:.2f} s")
```

In [ ]:
```
N_grande = 15000
```

In [ ]:
```
simular_agujero_negro(N_grande, 20)
```

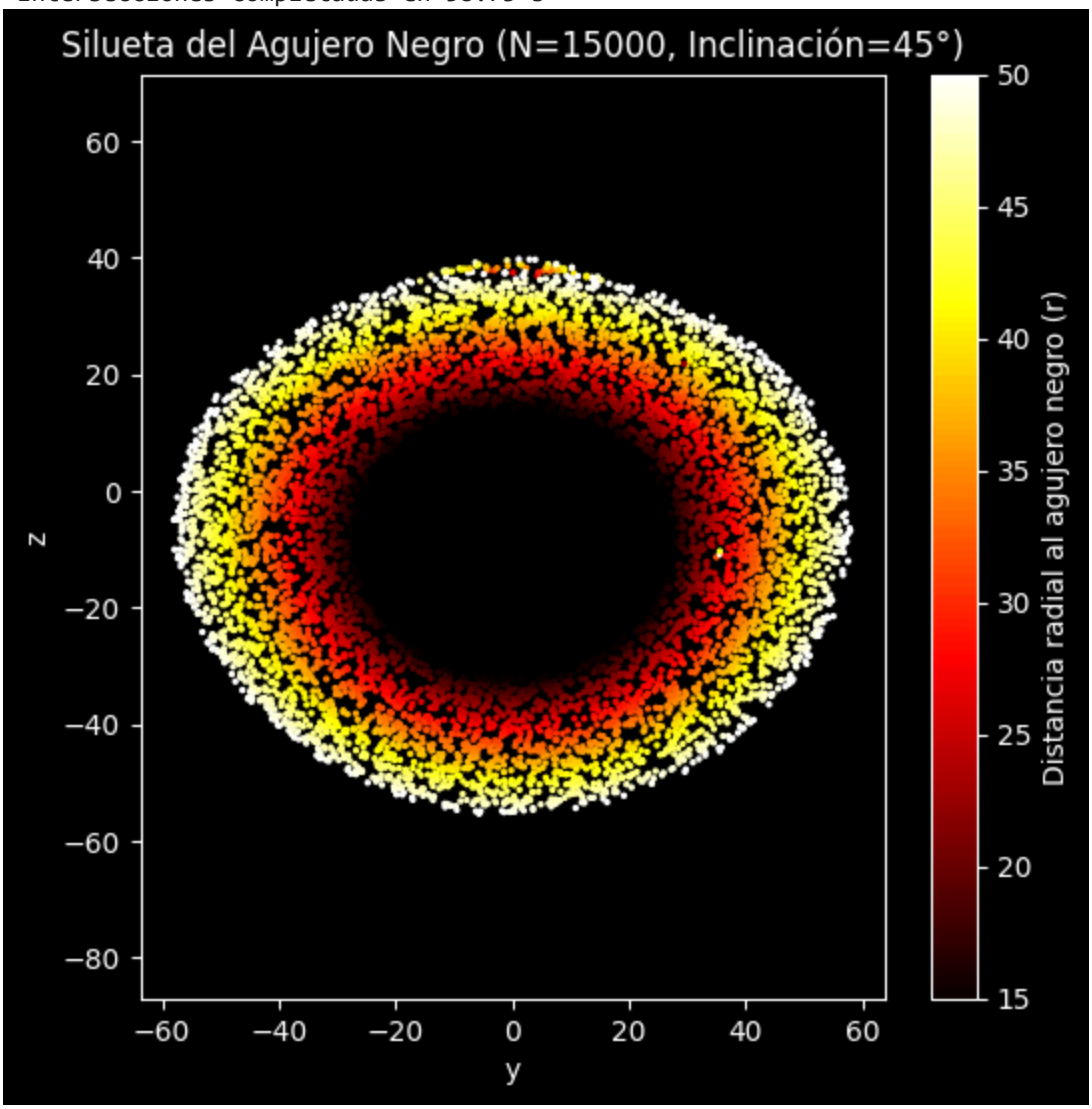
```
Simulación: N=15000, Inclinación=20°
/tmp/ipykernel_7300/3893832885.py:26: ODEintWarning: Excess work done on this call
(perhaps wrong Dfun type). Run with full_output = 1 to get quantitative information.
  sol = odeint(flow_deriv, estado_inicial, tspan)
/tmp/ipykernel_7300/3893832885.py:30: RuntimeWarning: overflow encountered in square
  r_traj = np.sqrt(x_traj**2 + y_traj**2)
/tmp/ipykernel_7300/3893832885.py:30: RuntimeWarning: invalid value encountered in s
quare
  r_traj = np.sqrt(x_traj**2 + y_traj**2)
Integración completada en 85.48 s
Intersecciones completadas en 21.66 s
```




```
Simulación total completada en 107.43 s
```

In [134…
```
simular_agujero_negro(N_grande, 45)
```

```
Simulación: N=15000, Inclinación=45°
/tmp/ipykernel_7300/3893832885.py:26: ODEintWarning: Excess work done on this call
(perhaps wrong Dfun type). Run with full_output = 1 to get quantitative information.
  sol = odeint(flow_deriv, estado_inicial, tspan)
/tmp/ipykernel_7300/3893832885.py:30: RuntimeWarning: overflow encountered in square
  r_traj = np.sqrt(x_traj**2 + y_traj**2)
Integración completada en 91.11 s
Intersecciones completadas en 58.73 s
```

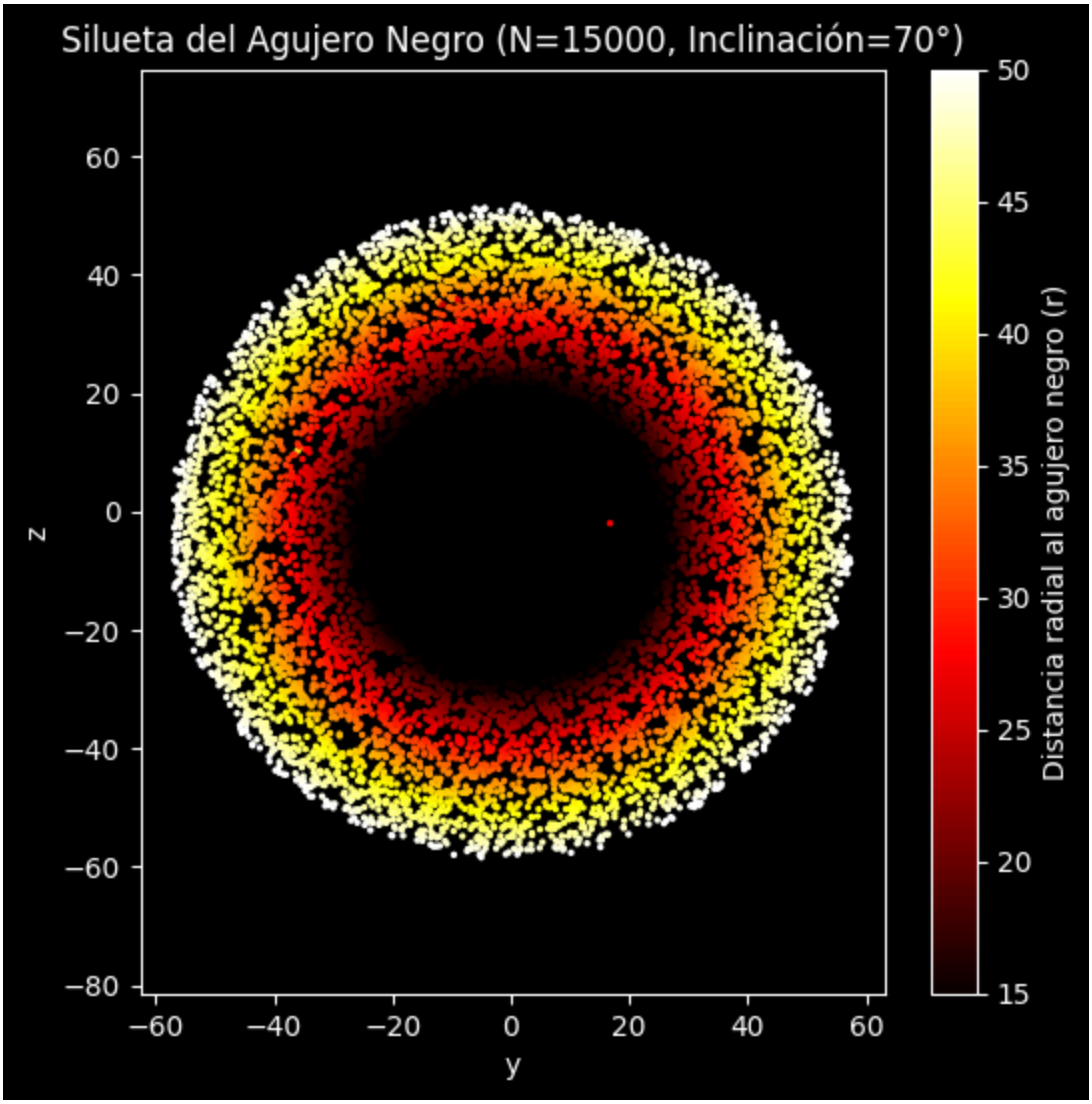



```
Simulación total completada en 150.38 s
```

In [135…

```
simular_agujero_negro(N_grande, 70)
```

```
Simulación: N=15000, Inclinación=70°
/tmp/ipykernel_7300/3893832885.py:26: ODEintWarning: Excess work done on this call
(perhaps wrong Dfun type). Run with full_output = 1 to get quantitative information.
  sol = odeint(flow_deriv, estado_inicial, tspan)
/tmp/ipykernel_7300/3893832885.py:30: RuntimeWarning: overflow encountered in square
  r_traj = np.sqrt(x_traj**2 + y_traj**2)
/tmp/ipykernel_7300/3893832885.py:30: RuntimeWarning: invalid value encountered in s
quare
  r_traj = np.sqrt(x_traj**2 + y_traj**2)
Integración completada en 86.18 s
Intersecciones completadas en 71.98 s
```




```
Simulación total completada en 158.52 s
```

Declaración del uso de IA: en la parte 5 del proyecto, el tiempo que se toma el código para producir las imágenes es bastante alto. Por eso, Gemini sugirió usar time (start_total = time.time(), etc.). Los códigos relacionados a time los hizo Gemini. Esto fue útil para verificar que no fuera un bucle infinito y que el código tuviera resultado. También para tener una idea de cuánto tiempo se toma el código en hacer cada parte.